Property Graphs

Master's degree in Data Science
Semantic Data Management
Lab Assignment

Authors:

Antoni Lopera Barril
Oriol Parera Cuscó

April 13, 2026

Contents

0 Introduction **2**

A Modeling, Loading, Evolving **2**
A.1 Modeling 2
A.2 Instantiating and Loading 3
A.3 Evolving the Graph 5

B Querying **6**

C Recommender **8**

D Graph Algorithms **10**

0 Introduction

This project aims to design and implement a robust graph database for academic publication data, leveraging the Neo4j platform. Utilizing raw data from *dblp*, a systen has been engineering that goes beyond tabular storage to facilitate complex structural dependencies and queries. The implementation follows a modular, automated approach to ensure reproducibility and scalability, taking into account the requirements from the statement.

Communication with Neo4j has been made through individual `.py` applications for each exercise, all maintaining the same structure of connecting to the driver and then executing the necessary queries of the section. For section A, subfolders contain the applications, in which under the `main` body one can specify: the connection URI from the instance, the user (set to "neo4j" as default), the password (set to "password123" for simplicity, but must be changed to the specific user credentials), and the target database ("neo4j" by default). Each folder from sections B, C, and D, contains a `db_connection.py` file that is then used in each application. These call `Neo4jApp` with the same 3 first arguments as before. If no `database` is specified as forth argument of the `Neo4jApp`, queries will be automatically executed in the default database.

A Modeling, Loading, Evolving

A.1 Modeling

The implementation begins with the design of a property graph schema, which clearly defines node labels, properties, and the relationships connecting them. Following, there is the table summing up all elements present in the database (table 1).

Table 1: Graph Data Model: Nodes and Edges

Category	Name	Properties / Details
Nodes	Author	name
	Paper	id, title, abstract, year, type
	Keyword	word
	Conference	name
	Edition	id, city, year
	Journal	name
	Volume	id, number, year
Edges	WRITTEN_BY (Paper → Author)	corresponding_author
	HAS_REVIEWED (Author → Paper)	–
	CITED_BY (Paper → Paper)	–
	COVERS (Paper → Keyword)	–
	PUBLISHED_IN (Paper → Volume)	–
	BELONGS_TO (Volume → Journal)	–
	PRESENTED_AT (Paper → Edition)	–
	BELONGS_TO (Edition → Conference)	–

Please note that for simplicity, since workshops and conferences share along all the project the same properties, no distinction will be made between those, and both will be collapsed into `Conference` nodes. Let us go into a detailed explanation of the graph schema, presented in figure 1. It will be composed of 7 node types: `Author` with its `name` as identifying property, each associated with at least one publication; `Paper` with `id`, `title`, `abstract`, `year`, and `type` indicating whether it comes from a proceeding (Inproceeding) or a journal (Article); `Keyword`, which includes different topics papers can talk about; `Conference`, with details about its `name`, and putting together workshops and conferences, as both are treated the exact same way; `Edition`, identified by `id`, and indicating the `city` and `year` in which the event was held; `Journal` including its `name`; and `Volume`, displaying as properties `id`, `number` and `year` of publication.

To complement this, a total of 8 relations will complete the property graph: `WRITTEN_BY` from papers to authors, `HAS_REVIEWED` from authors to papers, `CITED_BY` from papers to papers, `COVERS`

from papers to keywords, PRESENTED_AT from papers to editions, PUBLISHED_IN from papers to volumes, and BELONGS_TO. This last relation is not unique and will be shared between Volume or Edition and Journal or Conference respectively. Relations types such as one-to-many and many-to-many are displayed in the schema.

Constraints

Furthermore, there are also some intrinsic limitation of property graphs. Since certain constraints can be placed, one cannot limit certain relations such as the cardinality fact that each Paper must only have one BELONGS_TO edge, either to a Volume or an Edition, but not both at the same time, hence the XOR annotation that cannot be imposed in any other way.

Another vital rule is that an Author of a Paper linked through WRITTEN_BY cannot be at the same time linked with HAS_REVIEWED. Needless to say, somebody who has written a publication cannot be a reviewer of that same paper. Additionally, another detail is that only one corresponding_author property can be set to True per authorship relation.

On the other hand, a type of constraint that can be added owing to the data base management system being used, uniqueness constraints can indeed be added. Before conducting the data ingestion, a set of rules was applied to the schema. Primary identifiers were created for node entities (e.g., requiring the id of Paper and the name of Author to be unique). These guarantees data integrity, together with the automated MERGE operations, and fastens the label scans by means of indexed lookups, which is not necessary but definitely useful.

Justifications and Assumptions

Nonetheless, the most important part of the design is to be aware and justify the reasons for opting for this approach. First of all, **non-redundancy** and **maintenance** capability have been sought from the beginning. Hence, by separating Edition from Conference, and Volume from Journal, instead of putting the combination together in unique nodes, the schema avoids duplicating the parent's forum name across possibly thousands of papers. Consequently, updates are a single-node operation. Next, **reusability** has been considered. Putting Keyword in distinct nodes instead of listing keywords in the properties of Paper nodes allows multiple papers to be highly interconnected.

Apart from that, query requirements from this same project also must be considered. First, the existence of the relationship CITED_BY enables the counting of citations, and the connection with conferences or workshops are made by a direct traversal from it to Volume and eventually to Paper. In second place, to find authors with more than a threshold of publications on specific conventions, one can count the unique Edition nodes per Author by searching their written papers, and each of the former connects to the conference or workshop.

Lastly, impact factor and h-index must be evaluated for journals and authors for the third and fourth queries, respectively. Both are computed via citations, thus CITED_BY edges are of vital importance. These relations make possible aggregation, which in turn makes it easier to compute the aforementioned indicators.

A.2 Instantiating and Loading

As far as data gathering is concerned, the main source has been the computer science bibliography *dblp* [1]. From it, a .xml has been manually retrieved and further processed using Python. First of all, the tool given in [2] has been accessed and used in order to transform the .xml into multiple .csv containing all available data, making it easier to be read and processed, and eventually be ingested into the graph database using the Neo4j's LOAD CSV method.

To handle data transformation, generate CSV files detailing nodes and edges from the *dblp* output, and produce synthetic data, FormatCSV.py is used. Processed files are stored in the formatted_csv folder, and its content has to be moved to the import folder of the instance we are working with. Formatted CSV are included in the submission to avoid having to keep the *dblp* output files, which weight excessively. In contrast, UploadCSV.py uses the neo4j Python driver to programmatically instantiate the graph, to ensure a reproducible loading process, reading from the already-mentioned import folder.

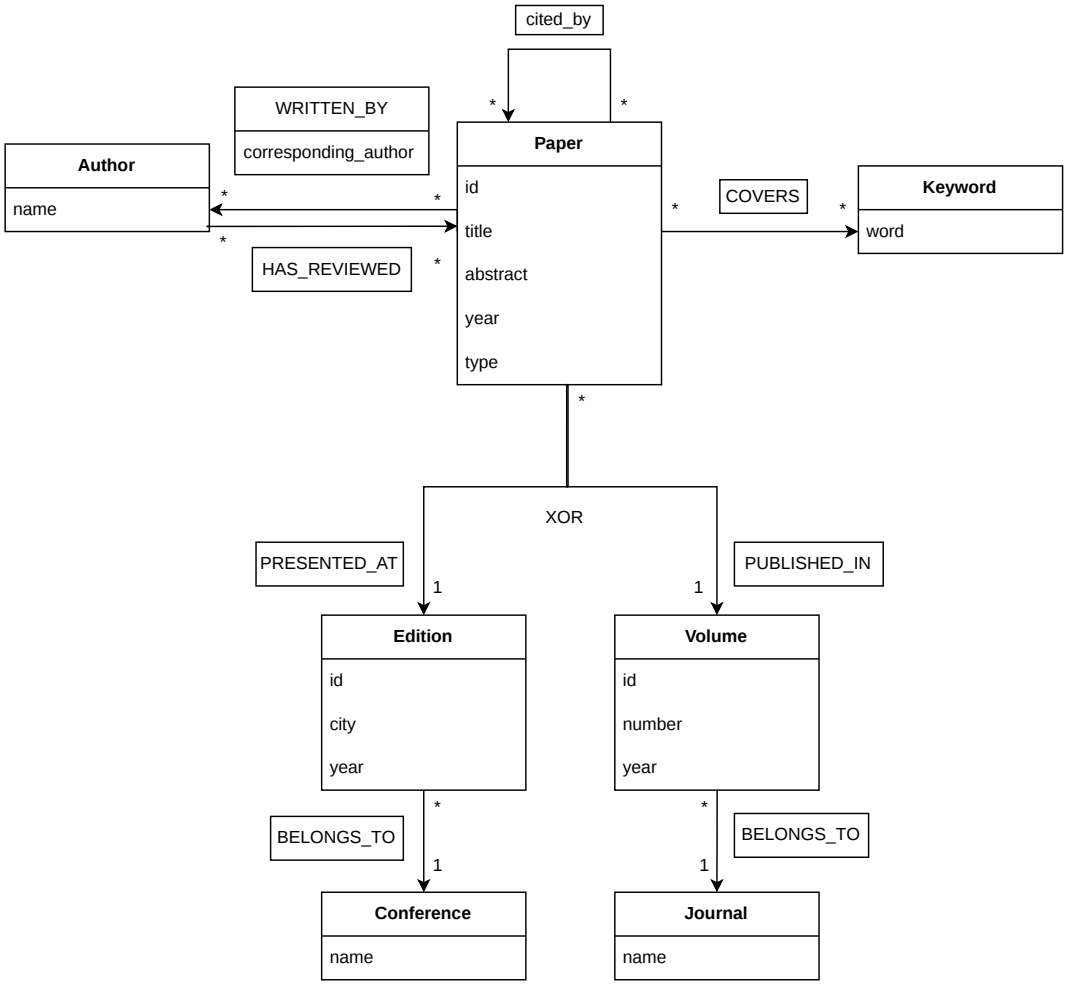

Figure 1: Graph database schema

However, not all resulting files have been used, only the ones containing papers from journals and the information on how the file's columns are distributed; the analogous but for the publications appearing in workshops' or conferences' proceedings; and the citation relations. The number of papers at our disposal are approximately 200,000 in total, which is an enormous amount.

Direct Ingestion

Taking the previous information into account, the preprocessing process could be started. To begin with, two subsets of 10,000 papers were chosen among articles and publications from proceedings, yielding a total of ∼60,000 nodes. This scale has been considered to be enough for the queries to be consistent, but the number of papers can be tuned by modifying the `n_papers` and `n_inproceedings` parameters in `FormatCSV.py`.

Directly from the *dblp* files, the extracted information was the article identifier, the journal where it was published and its volume and year of publication, which let us built the relation `PUBLISHED_IN`; and in case the paper was exposed in a forum, the volume of conference or workshop to relate with `PRESENTED_AT`, together with the year it was held; and the relations `BELONGS_TO`. Furthermore, authors could be directly extracted from the data and the boolean `corresponding_author` property of the `WRITTEN_BY` relations were set to `True` for the first author appearing in the contributors list. While initially it was planned to extract `CITED_BY` relations from the raw citations file, the subsampling process resulted in a very sparse citation network.

To address the previous and additional issues, artificially generated data has been considered.

Synthetic Data

As mentioned before, the first problem to be tackled is the lack of inter-citations between the subset, thus from 1 to 40 artificial relations of `CITED_BY` have been created per paper maintaining uniformity. This number has been chosen so that queries required for next sections give a wide range of possible outputs, regarding the Impact Factor, for instance. Moreover, topics covered in papers were not given in the sources, thus a pool of technical keywords has been manually created, including "data management", "indexing", "data modeling", "big data", "data processing", "data storage", "data querying", "query optimization", "nosql", "data mining", "computer vision", "machine learning", and "distributed systems". Then, each paper has been assigned from 2 to 5 randomly, originating relations of `COVERS`. Similarly, a pool of cities was created and one was assigned to the `city` property for all `Edition` nodes for forums. Initially, `Paper` had no abstracts, and these were generated following a template for all papers. As for reviewals, for each paper the potential authors to act as reviewers have been extracted, i.e. all authors that have not participated in that paper, and 3 of these have been randomly assigned to it for the `HAS_REVIEWED`.

A.3 Evolving the Graph

For the new features that must be implemented, the following decisions were made to leverage from powerful graph databases properties.

It was opted to extend the relation `HAS_REVIEWED` with two new properties, the string `content` and the boolean `decision`, rather than adding new nodes per each revision. This avoids unnecessary node creations and maintains traversal fast, as a reviewal is a relation strictly between one `Author` and one `Paper`. Each `Paper` node was also given an `accepted` property, indicating whether that paper has been positively reviewed by most of its reviewers. Rejected papers might have to be treated differently than accepted ones, such removing their connections of publication or presentation, but it has been decided to keep them as they are for simplicity. Related to this property, a new integer node property `required_reviewers` has been added to the `Journal` and `Conference` nodes, as these are static quantities from venues themselves.

The implementation of the modified schema is given in figure 2, and its details are as follows. First step was the addition of a new property `required_reviewers` to `Journal` and `Conference` nodes, and these were set to integers between 2 and 4 randomly. Then, `HAS_REVIEWED` relations were complemented with the properties `content`, generated artificially following a template, and `decision`, set to `True` (favourable) or `False` (not favourable) with a 75% probability of acceptance. Ultimately, these changes were propagated to `Paper` nodes. Their property `accepted` is only set to `True` if two

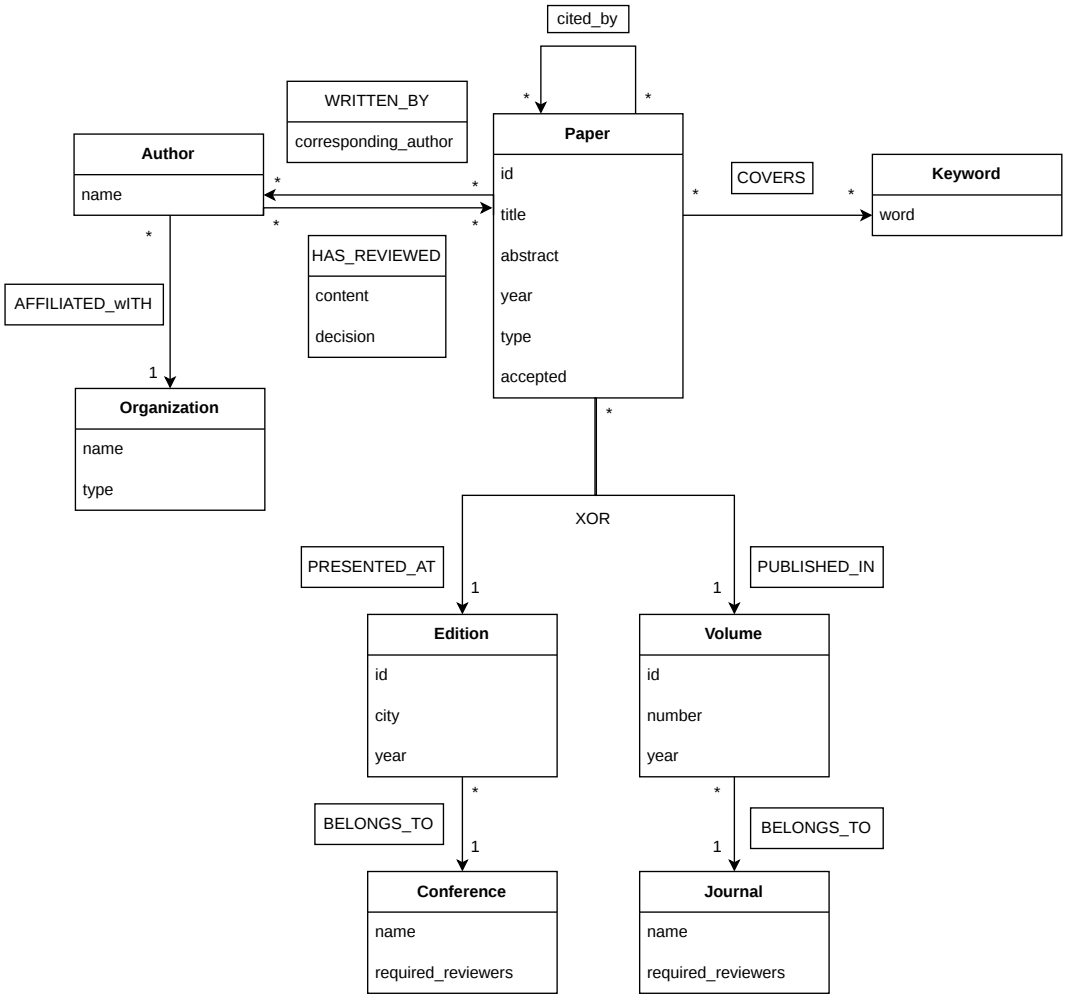

Figure 2: Graph database schema after evolution

conditions are met: the paper has been reviewed by more or exactly the number of required reviewers by their corresponding venue, and the majority of these have been positive, i.e. more than half of them.

Besides, another important additions are the `Organization` nodes, linked through the relation `AFFILIATED_WITH` coming from `Author` nodes, and as many authors will belong to the same institution, the relation will be many-to-one. Since affiliation data was absent from the source, it was synthetically simulated. A set of different organizations was manually inserted directly and authors were randomly affiliated to those through queries. It has been explicitly chosen to create nodes for each institution instead of adding properties to each author specifying what organization they belong to, to leverage the structural strengths of graph data bases. The main reasons are that, it would be easier to add specific information for each organization, such as location, to improve scalability without the need of updating all individual author records; maintains clean author profiles with only person-specific data; it allows for efficient graph traversals; and avoids redundancy by preventing the same institution name appearing repeatedly thousands of times.

B Querying

B1. Find the top 3 most cited papers of each conference/workshop.

This query identifies the most influential publications within each specific forum by calculating citation counts and storing them in descending order for all papers in each conference. To group results by

conference, it uses the `collect()` function to aggregate paper titles and their respective counts into a list, and slices these sorted lists to get the 3 most cited, rather than returning every paper in the database. Then, `UNWIND` let's us iterate over the list content. Output is each conference name, its 3 top papers' title, and their citations count.

```
// start by finding papers published by each conference and counting the number of
↪  citations they have
MATCH (c:Conference)<-[:BELONGS_TO]-(e:Edition)<-[:PRESENTED_AT]-(p1:Paper)
MATCH (p1)-[:CITED_BY]->(p2:Paper)
WITH c, p1, count(p2) as citations
ORDER BY citations DESC
// once ordered by citations in descending order, the first 3 results of the
↪  collected list will be the most cited ones for each conference
WITH c, collect({title: p1.title, count: citations})[0..3] as top3Papers
UNWIND top3Papers AS paper
RETURN c.name AS Conference, paper.title AS Title, paper.count AS Citations;
```

B2. For each conference/workshop find its community: i.e., those authors that have published papers on that conference/workshop in, at least, 4 different editions.

To define communities, the query traverses the graph from authors to papers, and then to the specific editions of conferences. The core of the logic relies on the `COUNT(DISTINCT e)`, ensuring unique events are taken into account rather than just total papers published. Only authors who have appeared in at least 4 different editions are returned, together with the corresponding conference.

```
// start by finding all inproceedings written by all authors and the corresponding
↪  conferences in which they have been presented
MATCH (p:Paper)-[:WRITTEN_BY]->(a:Author)
MATCH (p:Paper)-[:PRESENTED_AT]->(e:Edition)-[:BELONGS_TO]->(c:Conference)
// count how many different editions of conferences each author has presented at and
↪  check condition
WITH c, a, COUNT(DISTINCT e) as ndist
WHERE ndist >= 4
RETURN c.name, a.name;
```

B3. Find the impact factor of the journals in your graph.

This query calculates the prestige of journals within a specific temporal window. It first identifies the number of papers published in a journal during the 2-year period of 2023-2024, and then counts the citations of those specific papers received in the following year, 2025. The final impact factor (IF) is computed using the ratio between the two counts. The years could be changed to assess historical evolution of the impact factor. Results are sorted in descending IF order, including the journal name, and the two quantities used for the calculation.

```
MATCH (j:Journal)
// retrieve total papers and count from each journal from previous 2 years
OPTIONAL MATCH (p1:Paper)-[:PUBLISHED_IN]->(v1:Volume)-[:BELONGS_TO]->(j:Journal)
WHERE p1.year IN [2023,2024]
WITH j, collect(p1) as papers, count(p1) as total_papers
// compute total citations of papers published by each journal in actual year (using
↪  2025 as reference to have wider publication span)
UNWIND papers as p
OPTIONAL MATCH (p)-[:CITED_BY]->(p2:Paper)
WHERE p2.year = 2025
WITH j, total_papers, count(p2) as total_citations
// return computation of impact factor
RETURN j.name AS Journal,
       total_citations,
```

```
       total_papers,
       toFloat(total_citations) / total_papers AS ImpactFactor
ORDER BY ImpactFactor DESC;
```

B4. Find the h-index of the authors in your graph.

The H-index query measures an author's research impact. First, it collects all citation counts for an author's papers, sorting them in descending order. Then, it unwinds it on a numerical range to compare each paper's citation count against its position in the sorted list. This allows the algorithm to find the largest h such that the author has published h papers that have each been cited at least h times. This h-index is returned next to each author name.

```
// largest h such that h articles have at least h citations each
// match authors and their papers
MATCH (p:Paper)-[:WRITTEN_BY]->(a:Author)
// count total citations of each paper and each author and sort in descending order
OPTIONAL MATCH (p)-[:CITED_BY]->(citing:Paper)
WITH a, p, count(citing) AS citations
ORDER BY a.name, citations DESC
// collect counts of citations of each author in a list
WITH a, collect(citations) AS citationList
// h-index requires to compare value in list to its position
UNWIND range(0, size(citationList)-1) AS i
WITH a, citationList, i
WHERE citationList[i] >= (i + 1)
// return the maximum index that satisfied the condition
RETURN a.name AS Author, max(i + 1) AS HIndex
ORDER BY HIndex DESC;
```

C Recommender

C1. Community definition

After thoroughly assessing the requirements, the decisions to tackle this section have been the following. Communities have been declared as a new type of node. Since keywords were previously assigned, `Community` nodes could be linked to the corresponding `Keyword` nodes through a new relation `DEFINED_BY`, allowing for flexible many-to-many mapping. In this case, only the established *Database* community was declared, although the same approach may be followed to add any other. The analogy is made possible because communities have been defined as nodes instead of simple properties, enhancing performance of community hub analysis, and addition of scalable community properties.

```
MERGE (c:Community {name: "Database"})
WITH c
UNWIND ["data management", "indexing", "data modeling", "big data", "data
↪  processing", "data storage", "data querying"] AS db_keywords
MATCH (k:Keyword {word: db_keywords})
MERGE (c)-[:DEFINED_BY]->(k)
RETURN c.name AS Community, collect(k.word) AS LinkedKeywords // optional return
↪  sequence to check
```

C2. Venue relation with communities

In second place, all `Conference` and `Journal` nodes related to the aforementioned community have been identified using the query below, applying the 90% threshold, and related to `Community` by the `RELATED_TO` edges. This choice minimizes intermediate results and disk accesses when filtering papers by community for the subsequent steps.

```
MATCH (com:Community {name: "Database"})-[:DEFINED_BY]->(kw:Keyword) // finds
↪  corresponding keywords
WITH com, collect(kw) AS communityKeywords // groups those into a list for
↪  comparison
MATCH (p:Paper)-[:PRESENTED_AT|PUBLISHED_IN]->()-[:BELONGS_TO]->(v)
WHERE v:Conference OR v:Journal
WITH com, v, communityKeywords, collect(DISTINCT p) AS allPapers // papers ever
↪  participating in that venue
WITH com, v, size(allPapers) AS total,
     [paper IN allPapers WHERE EXISTS { (paper)-[:COVERS]->(k) WHERE k IN
     ↪  communityKeywords }] AS communityPapers // papers that actually cover at
     ↪  least one topic related to community
WITH com, v, total, size(communityPapers) AS countComm
WHERE total > 0 AND (toFloat(countComm) / total) >= 0.9 // apply threshold
MERGE (v)-[:RELATED_TO]->(com) // create relationship
RETURN v.name AS Venue, total AS TotalPapers, countComm AS CommunityPapers; //
↪  optional return sequence to check
```

C3. Top papers from each community venue

Thirdly, to identify the top 100 publications of the community-related venues, after they have been retrieved by the query output, it is enough to mark them with a new label, `TopPaper`, which will avoid having to re-calculate complex citation counts, specially for the next stage, as well as fulfilling the requirement of asserting inferred knowledge back into the graph. Apart from that, these nodes have been marked with a property of `isTopPaper` equal to `True`, otherwise set to `False`.

```
MATCH (v)-[:RELATED_TO]->(c:Community {name: "Database"})
// p1 is the paper IN the database venue (The one we want to label)
MATCH (p1:Paper)-[:PRESENTED_AT|PUBLISHED_IN]->()-[:BELONGS_TO]->(v)
// p2 is any paper citing p1
MATCH (p1)-[:CITED_BY]->(p2:Paper)
// Ensure the citations come from the wider database community
WHERE EXISTS {
↪  (p2)-[:PRESENTED_AT|PUBLISHED_IN]->()-[:BELONGS_TO]->()-[:RELATED_TO]->(c) }
WITH p1, count(p2) AS citations
ORDER BY citations DESC
LIMIT 100
SET p1:TopPaper, p1.isTop100 = true
RETURN p1.title AS Title, citations AS Citations;
```

C4. Potential reviewers and gurus

Lastly, leveraging from the newly created node label for the 100 papers with the highest number of citations among the community, their `Author` can be easily extracted. Then, these nodes are marked with another label `PotentialReviewer`. Furhermore, if these same authors have 2 or more papers among these 100, they are also assigned the `Guru` label. This multi-labelling approach could have been encoded as simple node properties, but this way it is facilitated to find these potential good matches and reputable authors. Note this might lead to nodes having more than 1 label at a time, but this is not an unforeseen problem.

```
MATCH (p:TopPaper)-[:WRITTEN_BY]->(a:Author)
WITH a, count(p) AS writtenPapers
SET a:PotentialReviewer
WITH a, writtenPapers
WHERE writtenPapers >= 2
SET a:Guru
RETURN a.name AS Author, writtenPapers;
```

D Graph Algorithms

In order to go beyond simple query limits, one must consider the wide range of powerful graph algorithms to compute domain-specific useful metrics. The following queries of the Graph Data Science (GDS) library are very similar as far as their structure is concerned: first the graph, nodes, and relationships of interest to be analysed, are projected as subgraphs into the server's RAM; and another query executes the algorithm on this projected in-memory graph.

D1. PageRank algorithm

For this section, our focus will be on two specific algorithms, starting with *PageRank*. This is a centrality algorithm which relies on the quantity and quality of the incoming edges to find the importance or relevance of nodes. In the domain of research publications, a simple citation count will treat all citations as equal, regardless of the number of citations of the citing paper, for instance. However, instead of this simple analysis, *PageRank* consider a paper highly influential if it is cited by other papers that are in turn highly influential, yielding to the papers that have served as foundation for the field. The syntax to use is the following.

```
CALL gds.graph.project(
  'citationGraph',
  'Paper',
  'CITED_BY'
);

CALL gds.pageRank.stream('citationGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).title AS Paper, score
ORDER BY score DESC
LIMIT 10;
```

The data obtained as output is a numerical score for each `Paper` node, which has been limited to 10 rows for simplicity. Higher scores indicate a more primordial publication inside the environment. To get a better understanding, it can be regarded as somehow the likelihood of landing on that specific node while randomly traversing the graph.

D2. Louvain algorithm

The second algorithm that will be considered is *Louvain*. This method maximizes modularity, a scale that measures density of links inside a community versus links between communities. Inside the context of research publications, it is used to identify groups of authors that cite each other much more frequently than the rest of the network. This might be due to strong collaboration relationships or similar work areas. Commonly this leads to merge smaller and well-defined communities into larger ones if this measurement is benefitted. Syntax to call this method is as follows.

```
CALL gds.graph.project(
  'collaborationGraph',
  ['Author', 'Paper'],
  {
    WRITTEN_BY: {
      orientation: 'UNDIRECTED'
    }
  }
);

CALL gds.louvain.stream('collaborationGraph')
YIELD nodeId, communityId
WHERE 'Author' IN labels(gds.util.asNode(nodeId))
RETURN communityId, collect(gds.util.asNode(nodeId).name) AS Researchers
ORDER BY size(Researchers) DESC
LIMIT 5;
```

The resulting output data is a co-authorship study in the form of research clusters, limited to 5 for readability. It returns two columns, one for the community identifier, i.e. cluster identifier, and the second one is a list of the author names that belong to that group.

In this synthetic environment, where citations are distributed without any bias, the algorithm fails to distinguish boundaries and leads to fewer and much larger communities, rather than the fragmented, specialized sub-fields typically observed in real-word networks. In spite of this resulting a challenge, the general ideas that can be extracted for a real case are clear.

References

[1] dblp Team. *dblp: computer science bibliography*. 1993. URL: https://dblp.org.

[2] Thom Hurks. *dblp-to-csv*. Version v1.0.0. 2020. URL: https://github.com/ThomHurks/dblp-to-csv.